

3.9: Common Table Expressions

Ben Diemler

Step 1:

Subquery Step 1:

```
SELECT AVG(total_amount_paid) AS
avg_payment_amount
FROM
(SELECT A.customer_id,
A.first_name,
A.last_name,
C.city,
D.country,
SUM(amount) AS total_amount_paid
FROM payment E
INNER JOIN customer A ON E.customer_id =
A.customer_id
INNER JOIN address B ON A.address_id =
B.address_id
INNER JOIN city C ON B.city_id = C.city_id
INNER JOIN country D ON C.country_id =
D.country_id
WHERE city IN ('Aurora', 'Atlixco', 'Xintai', 'Adoni',
'Dhule (Dhulia)',
'Kurashiki', 'Pingxiang', 'Sivas', 'Celaya', 'So Leopoldo')
GROUP BY C.city, A.customer_id, A.first_name,
A.last_name, D.country
ORDER BY total_amount_paid DESC
LIMIT 5) AS average
```

CTE:

```
WITH avg_payment_amount_cte
AS
(SELECT A.customer_id,
A.first_name,
A.last_name,
C.city,
D.country,
SUM(amount) AS total_amount_paid
FROM payment E
INNER JOIN customer A ON E.customer_id =
A.customer_id
INNER JOIN address B ON A.address_id = B.address_id
INNER JOIN city C ON B.city_id = C.city_id
INNER JOIN country D ON C.country_id = D.country_id
WHERE city IN ('Aurora', 'Atlixco', 'Xintai', 'Adoni', 'Dhule
(Dhulia)',
'Kurashiki', 'Pingxiang', 'Sivas', 'Celaya', 'So Leopoldo')
GROUP BY C.city, A.customer_id, A.first_name,
A.last_name, D.country
ORDER BY total_amount_paid DESC
LIMIT 5)
SELECT AVG(total_amount_paid) AS
avg_payment_amount
FROM avg_payment_amount_cte;
```

```
Query History
1 WITH avg_payment_amount_cte
2 AS
3 (SELECT A.customer_id,
4 A.first_name,
5 A.last_name,
6 C.city,
7 D.country,
8 SUM(amount) AS total_amount_paid
9 FROM payment E
10 INNER JOIN customer A ON E.customer_id = A.customer_id
11 INNER JOIN address B ON A.address_id = B.address_id
12 INNER JOIN city C ON B.city_id = C.city_id
13 INNER JOIN country D ON C.country_id = D.country_id
14 WHERE city IN ('Aurora', 'Atlixco', 'Xintai', 'Adoni', 'Dhule (Dhulia)',
15 'Kurashiki', 'Pingxiang', 'Sivas', 'Celaya', 'So Leopoldo')
16 GROUP BY C.city, A.customer_id, A.first_name, A.last_name, D.country
17 ORDER BY total_amount_paid DESC
18 LIMIT 5)
19 SELECT AVG(total_amount_paid) AS avg_payment_amount
20 FROM avg_payment_amount_cte;
21
22
23
24
25
```

Data Output Messages Notifications

	avg_payment_amount
1	107.3540000000000000

Subquery Step 2:

```
SELECT ctr.country,
COUNT (DISTINCT cst.customer_id) AS
all_customer_count,
COUNT(DISTINCT top_5_customers.customer_id) AS
top_customer_count
FROM customer AS cst
INNER JOIN address AS adr ON cst.address_id =
adr.address_id
INNER JOIN city AS cty ON adr.city_id = cty.city_id
INNER JOIN country AS ctr ON cty.country_id =
ctr.country_id
LEFT JOIN
(SELECT A.customer_id,
A.first_name,
A.last_name,
C.city,
D.country,
SUM(amount) AS total_amount_paid
FROM payment E
INNER JOIN customer A ON E.customer_id =
A.customer_id
INNER JOIN address B ON A.address_id = B.address_id
INNER JOIN city C ON B.city_id = C.city_id
INNER JOIN country D ON C.country_id = D.country_id
WHERE city IN ('Aurora', 'Atlixco', 'Xintai', 'Adoni', 'Dhule
(Dhulia)',
'Kurashiki', 'Pingxiang', 'Sivas', 'Celaya', 'So Leopoldo')
GROUP BY C.city, A.customer_id, A.first_name,
A.last_name, D.country
ORDER BY total_amount_paid DESC
LIMIT 5)
AS top_5_customers
ON top_5_customers.country = ctr.country
GROUP BY ctr.country
ORDER BY top_customer_count DESC,
all_customer_count DESC
```

CTE:

```
WITH top_customer_count_CTE
AS
(SELECT A.customer_id,
A.first_name,
A.last_name,
C.city,
D.country,
SUM(amount) AS total_amount_paid
FROM payment E
INNER JOIN customer A ON E.customer_id =
A.customer_id
INNER JOIN address B ON A.address_id = B.address_id
INNER JOIN city C ON B.city_id = C.city_id
INNER JOIN country D ON C.country_id = D.country_id
WHERE city IN ('Aurora', 'Atlixco', 'Xintai', 'Adoni', 'Dhule
(Dhulia)',
'Kurashiki', 'Pingxiang', 'Sivas', 'Celaya', 'So Leopoldo')
GROUP BY C.city, A.customer_id, A.first_name,
A.last_name, D.country
ORDER BY total_amount_paid DESC
LIMIT 5)
SELECT ctr.country,
COUNT (DISTINCT cst.customer_id) AS
all_customer_count,
COUNT(DISTINCT
top_customer_count_CTE.customer_id) AS
top_customer_count
FROM customer AS cst
INNER JOIN address AS adr ON cst.address_id =
adr.address_id
INNER JOIN city AS cty ON adr.city_id = cty.city_id
INNER JOIN country AS ctr ON cty.country_id =
ctr.country_id
LEFT JOIN top_customer_count_CTE ON
top_customer_count_CTE.country = ctr.country
GROUP BY ctr.country
ORDER BY top_customer_count DESC,
all_customer_count DESC
```

Query Query History

```
1 WITH top_customer_count_CTE
2 AS
3 (SELECT A.customer_id,
4 A.first_name,
5 A.last_name,
6 C.city,
7 D.country,
8 SUM(amount) AS total_amount_paid
9 FROM payment E
10 INNER JOIN customer A ON E.customer_id = A.customer_id
11 INNER JOIN address B ON A.address_id = B.address_id
12 INNER JOIN city C ON B.city_id = C.city_id
13 INNER JOIN country D ON C.country_id = D.country_id
14 WHERE city IN ('Aurora', 'Atlixco', 'Xintai', 'Adoni', 'Dhule (Dhulia)',
15 'Kurashiki', 'Pingxiang', 'Sivas', 'Celaya', 'So Leopoldo')
16 GROUP BY C.city, A.customer_id, A.first_name, A.last_name, D.country
17 ORDER BY total_amount_paid DESC
18 LIMIT 5)
19 SELECT ctr.country,
20 COUNT (DISTINCT cst.customer_id) AS all_customer_count,
21 COUNT(DISTINCT top_customer_count_CTE.customer_id) AS top_customer_count
22 FROM customer AS cst
23 INNER JOIN address AS adr ON cst.address_id = adr.address_id
24 INNER JOIN city AS cty ON adr.city_id = cty.city_id
25 INNER JOIN country AS ctr ON cty.country_id = ctr.country_id
```

Data Output Messages Notifications

SQL

country	all_customer_count	top_customer_count
character varying (50)	bigint	bigint
1 Mexico	30	2
2 India	60	1
3 United States	36	1
4 Turkey	15	1

3- To write the CTE, my first step was to find and compare the subqueries. After that, I would reference back to the lesson to see how the subquery would convert into CTE. Considering that the Query that produced by CTE was my subquery, I first added the subquery to correct location and worked slowly around it, substituting what was needed. As I had many errors throughout my testing phase, I also got AI help and checked other tasks to identify where my mistake was.

Step 2:

1- Without checking costs, I believe that the CTE will perform better, considering that it looks like a more detailed query, which makes me believe that it has more specific information to retrieve information, and is not a combination of query + subqueries running together.

2- Costs analysis using EXPLAIN on each query above

Query 1: Average amount paid by top 5 csts:

Subquery:

Query	Query History
1	EXPLAIN
2	SELECT AVG(total_amount_paid) AS avg_payment_amount
3	FROM
4	(SELECT A.customer_id,
5	A.first_name,
6	A.last_name,
7	C.city,
8	D.country,
9	SUM(amount) AS total_amount_paid
10	FROM payment E
11	INNER JOIN customer A ON E.customer_id = A.customer_id
12	INNER JOIN address B ON A.address_id = B.address_id
13	INNER JOIN city C ON B.city_id = C.city_id
14	INNER JOIN country D ON C.country_id = D.country_id
15	WHERE city IN ('Aurora', 'Atlixco', 'Xintei', 'Adoni', 'Dhule (Dhulia)',
16	'Kureshiki', 'Pingxiang', 'Sivas', 'Celaya', 'So Leopoldo')
17	GROUP BY C.city, A.customer_id, A.first_name, A.last_name, D.country
18	ORDER BY total_amount_paid DESC
19	LIMIT 5) AS average
20	
21	
22	
23	
24	
25	
26	
27	
28	
29	
30	

Data Output	Messages	Notifications
                SQL		
QUERY PLAN		
text		
1	Aggregate	(cost=64.45..64.46 rows=1 width=32)
2	→ Limit	(cost=64.37..64.39 rows=5 width=67)
3	→ Sort	(cost=64.37..64.98 rows=243 width=67)
4	Sort Key:	(sum(e.amount)) DESC
5	→ HashAggregate	(cost=57.30..60.34 rows=243 width=67)
6	Group Key:	c.city, a.customer_id, d.country
7	→ Nested Loop	(cost=18.16..54.87 rows=243 width=41)
8	→ Hash Join	(cost=17.88..37.14 rows=10 width=35)
9	Hash Cond:	(c.country_id = d.country_id)
10	→ Nested Loop	(cost=14.43..33.66 rows=10 width=28)
11	→ Hash Join	(cost=14.15..29.77 rows=10 width=15)
12	Hash Cond:	(b.city_id = c.city_id)
13	→ Seq Scan on address b	(cost=0.00..14.03 rows=6..)
14	→ Hash	(cost=14.03..14.03 rows=10 width=15)
15	→ Seq Scan on city c	(cost=0.00..14.03 rows=10..)
16	Filter:	((city)::text = ANY ({Aurora,Atlixco,Xinta..
17	→ Index Scan using idx_fk_address_id on customer a (c..)	
18	Index Cond:	(address_id = b.address_id)
19	→ Hash	(cost=2.09..2.09 rows=109 width=13)
20	→ Seq Scan on country d	(cost=0.00..2.09 rows=109 wi..)
21	→ Index Scan using idx_fk_customer_id on payment e	(cost=0..)
22	Index Cond:	(customer_id = a.customer_id)

Total rows: 22 Query complete 00:00:00.079

CTE:

Query	Query History
1	EXPLAIN
2	WITH avg_payment_amount_cte
3	AS
4	(SELECT A.customer_id,
5	A.first_name,
6	A.last_name,
7	C.city,
8	D.country,
9	SUM(amount) AS total_amount_paid
10	FROM payment E
11	INNER JOIN customer A ON E.customer_id = A.customer_id
12	INNER JOIN address B ON A.address_id = B.address_id
13	INNER JOIN city C ON B.city_id = C.city_id
14	INNER JOIN country D ON C.country_id = D.country_id
15	WHERE city IN ('Aurora', 'Atlixco', 'Xintei', 'Adoni', 'Dhule (Dhulia)',
16	'Kureshiki', 'Pingxiang', 'Sivas', 'Celaya', 'So Leopoldo')
17	GROUP BY C.city, A.customer_id, A.first_name, A.last_name, D.country
18	ORDER BY total_amount_paid DESC
19	LIMIT 5)
20	SELECT AVG(total_amount_paid) AS avg_payment_amount
21	FROM avg_payment_amount_cte;
22	
23	
24	
25	
26	
27	
28	
29	
30	
31	

Data Output	Messages	Notifications
                SQL		
QUERY PLAN		
text		
1	Aggregate	(cost=64.45..64.46 rows=1 width=32)
2	→ Limit	(cost=64.37..64.39 rows=5 width=67)
3	→ Sort	(cost=64.37..64.98 rows=243 width=67)
4	Sort Key:	(sum(e.amount)) DESC
5	→ HashAggregate	(cost=57.30..60.34 rows=243 width=67)
6	Group Key:	c.city, a.customer_id, d.country
7	→ Nested Loop	(cost=18.16..54.87 rows=243 width=41)
8	→ Hash Join	(cost=17.88..37.14 rows=10 width=35)
9	Hash Cond:	(c.country_id = d.country_id)
10	→ Nested Loop	(cost=14.43..33.66 rows=10 width=28)
11	→ Hash Join	(cost=14.15..29.77 rows=10 width=15)
12	Hash Cond:	(b.city_id = c.city_id)
13	→ Seq Scan on address b	(cost=0.00..14.03 rows=6..)
14	→ Hash	(cost=14.03..14.03 rows=10 width=15)
15	→ Seq Scan on city c	(cost=0.00..14.03 rows=10..)
16	Filter:	((city)::text = ANY ({Aurora,Atlixco,Xinta..
17	→ Index Scan using idx_fk_address_id on customer a (c..)	
18	Index Cond:	(address_id = b.address_id)
19	→ Hash	(cost=2.09..2.09 rows=109 width=13)
20	→ Seq Scan on country d	(cost=0.00..2.09 rows=109 wi..)
21	→ Index Scan using idx_fk_customer_id on payment e	(cost=0..)
22	Index Cond:	(customer_id = a.customer_id)

Total rows: 22 Query complete 00:00:00.081

Query 2: Top 5 csts per country

Subquery:

Query	Query History
3	COUNT (DISTINCT cst.customer_id) AS all_customer_count,
4	COUNT(DISTINCT top_5_customers.customer_id) AS top_customer_count
5	FROM customer AS cst
6	INNER JOIN address AS adr ON cst.address_id = adr.address_id
7	INNER JOIN city AS cty ON adr.city_id = cty.city_id
8	INNER JOIN country AS ctr ON cty.country_id = ctr.country_id
9	LEFT JOIN
10	(SELECT A.customer_id,
11	A.first_name,
12	A.last_name,
13	C.city,
14	D.country,
15	SUM(amount) AS total_amount_paid
16	FROM payment E
17	INNER JOIN customer A ON E.customer_id = A.customer_id
18	INNER JOIN address B ON A.address_id = B.address_id
19	INNER JOIN city C ON B.city_id = C.city_id

Data Output Messages Notifications

SQL

QUERY PLAN
text

1

Sort (cost=168.72..168.99 rows=109 width=25)

2

Sort Key: (count(DISTINCT top_5_customers.customer_id)) DESC, (count(DISTINCT top_5_customers.customer_id))

3

→ GroupAggregate (cost=157.95..165.03 rows=109 width=25)

4

Group Key: ctr.country

5

→ Sort (cost=157.95..159.45 rows=599 width=17)

6

Sort Key: ctr.country, cst.customer_id

7

→ Hash Left Join (cost=108.02..130.32 rows=599 width=17)

8

Hash Cond: ((ctr.country)::text = (top_5_customers.country)::text)

9

→ Hash Join (cost=43.52..63.30 rows=599 width=13)

10

Hash Cond: (cty.country_id = ctr.country_id)

11

→ Hash Join (cost=40.07..58.22 rows=599 width=6)

12

Hash Cond: (adr.city_id = cty.city_id)

13

→ Hash Join (cost=21.57..38.14 rows=599 width=6)

14

Hash Cond: (cst.address_id = adr.address_id)

15

→ Seq Scan on customer cst (cost=0.00..14.99 rows=599 width=6)

16

→ Hash (cost=14.03..14.03 rows=603 width=6)

17

→ Seq Scan on address adr (cost=0.00..14.03 rows=599 width=6)

18

→ Hash (cost=11.00..11.00 rows=600 width=6)

19

→ Seq Scan on city cty (cost=0.00..11.00 rows=600 width=6)

20

→ Hash (cost=2.09..2.09 rows=109 width=13)

21

→ Seq Scan on country ctr (cost=0.00..2.09 rows=109 width=6)

22

→ Hash (cost=64.44..64.44 rows=5 width=13)

23

→ Subquery Scan on top_5_customers (cost=64.37..64.44 rows=5 width=13)

24

→ Limit (cost=64.37..64.39 rows=5 width=67)

25

→ Sort (cost=64.37..64.98 rows=243 width=67)

26

Sort Key: (sum(a.amount)) DESC

27

→ HashAggregate (cost=57.30..60.34 rows=243 width=13)

28

Group Key: c.city, a.customer_id, d.country

29

→ Nested Loop (cost=18.16..54.87 rows=243 width=13)

30

→ Hash Join (cost=17.88..37.14 rows=10 width=6)

31

Hash Cond: (c.country_id = d.country_id)

32

→ Nested Loop (cost=14.43..33.66 rows=10 width=6)

33

→ Hash Join (cost=14.15..29.77 rows=10 width=6)

34

Hash Cond: (b.city_id = c.city_id)

35

→ Seq Scan on address b (cost=0.00..14.03 rows=599 width=6)

36

→ Hash (cost=14.03..14.03 rows=603 width=6)

37

→ Seq Scan on city c (cost=0.00..11.00 rows=600 width=6)

38

Filter: ((city)::text = ANY ({Aurora, Atlitico, Xintei, Adoni, Dhule (Dhulia), Kureshiki, Pingxiang, Sivas, Celaya, So Leopoldo}))

39

→ Index Scan using idx_fk_address_id on payment E (cost=0.00..14.03 rows=599 width=6)

40

Index Cond: (address_id = b.address_id)

41

→ Hash (cost=2.09..2.09 rows=109 width=13)

42

→ Seq Scan on country d (cost=0.00..2.09 rows=109 width=6)

43

→ Index Scan using idx_fk_customer_id on payment E (cost=0.00..14.03 rows=599 width=6)

44

Index Cond: (customer_id = a.customer_id)

Total rows: 44 Query complete 00:00:00.075

CTE:

Query	Query History
1	EXPLAIN
2	WITH top_customer_count_CTE
3	AS
4	(SELECT A.customer_id,
5	A.first_name,
6	A.last_name,
7	C.city,
8	D.country,
9	SUM(amount) AS total_amount_paid
10	FROM payment E
11	INNER JOIN customer A ON E.customer_id = A.customer_id
12	INNER JOIN address B ON A.address_id = B.address_id
13	INNER JOIN city C ON B.city_id = C.city_id
14	INNER JOIN country D ON C.country_id = D.country_id
15	WHERE city IN ('Aurora', 'Atlixco', 'Xintei', 'Adoni', 'Dhule (Dhulia)', 'Kureshiki', 'Pingxiang', 'Sivas', 'Celaya', 'So Leopoldo')
17	GROUP BY C.city, A.customer_id, A.first_name, A.last_name, D.country

Data Output Messages Notifications

SQL

QUERY PLAN
text

1

Sort (cost=168.72..168.99 rows=109 width=25)

2

Sort Key: (count(DISTINCT top_customer_count_cte.customer_id)) DESC, (count(DISTINCT top_customer_count_cte.customer_id))

3

→ GroupAggregate (cost=157.95..165.03 rows=109 width=25)

4

Group Key: ctr.country

5

→ Sort (cost=157.95..159.45 rows=599 width=17)

6

Sort Key: ctr.country, cst.customer_id

7

→ Hash Left Join (cost=108.02..130.32 rows=599 width=17)

8

Hash Cond: ((ctr.country)::text = (top_customer_count_cte.country)::text)

9

→ Hash Join (cost=43.52..63.30 rows=599 width=13)

10

Hash Cond: (cty.country_id = ctr.country_id)

11

→ Hash Join (cost=40.07..58.22 rows=599 width=6)

12

Hash Cond: (adr.city_id = cty.city_id)

13

→ Hash Join (cost=21.57..38.14 rows=599 width=6)

14

Hash Cond: (cst.address_id = adr.address_id)

15

→ Seq Scan on customer cst (cost=0.00..14.99 rows=599 width=6)

16

→ Hash (cost=14.03..14.03 rows=603 width=6)

17

→ Seq Scan on address adr (cost=0.00..14.03 rows=599 width=6)

18

→ Hash (cost=11.00..11.00 rows=600 width=6)

19

→ Seq Scan on city cty (cost=0.00..11.00 rows=600 width=6)

20

→ Hash (cost=2.09..2.09 rows=109 width=13)

21

→ Seq Scan on country ctr (cost=0.00..2.09 rows=109 width=6)

22

→ Hash (cost=64.44..64.44 rows=5 width=13)

23

→ Subquery Scan on top_customer_count_cte (cost=64.37..64.44 rows=5 width=13)

24

→ Limit (cost=64.37..64.39 rows=5 width=67)

25

→ Sort (cost=64.37..64.98 rows=243 width=67)

26

Sort Key: (sum(a.amount)) DESC

27

→ HashAggregate (cost=57.30..60.34 rows=243 width=13)

28

Group Key: c.city, a.customer_id, d.country

29

→ Nested Loop (cost=18.16..54.87 rows=243 width=13)

30

→ Hash Join (cost=17.88..37.14 rows=10 width=6)

31

Hash Cond: (c.country_id = d.country_id)

32

→ Nested Loop (cost=14.43..33.66 rows=10 width=6)

33

→ Hash Join (cost=14.15..29.77 rows=10 width=6)

34

Hash Cond: (b.city_id = c.city_id)

35

→ Seq Scan on address b (cost=0.00..14.03 rows=599 width=6)

36

→ Hash (cost=14.03..14.03 rows=603 width=6)

37

→ Seq Scan on city c (cost=0.00..11.00 rows=600 width=6)

38

Filter: ((city)::text = ANY ({Aurora, Atlitico, Xintei, Adoni, Dhule (Dhulia), Kureshiki, Pingxiang, Sivas, Celaya, So Leopoldo}))

39

→ Index Scan using idx_fk_address_id on payment E (cost=0.00..14.03 rows=599 width=6)

40

Index Cond: (address_id = b.address_id)

41

→ Hash (cost=2.09..2.09 rows=109 width=13)

42

→ Seq Scan on country d (cost=0.00..2.09 rows=109 width=6)

43

→ Index Scan using idx_fk_customer_id on payment E (cost=0.00..14.03 rows=599 width=6)

44

Index Cond: (customer_id = a.customer_id)

Total rows: 44 Query complete 00:00:00.088

	Subquery	Cost
Query 1 cost	64.45	64.45
Query 1 Est time	00:00:00.078	00:00:00.081
Query 2 cost	168.72	168.72
Query 2 Est time	00:00:00.075	00:00:00.088

Yes, the results surprised me a little, as I thought CTEs would run faster. Although both had the same cost, and same results on the query plan, the CTEs did run faster. I will point out though, that I ran the explain controls more than once, and the estimated time could change from one attempt to another, so this would be just an estimate.

It does make sense though that CTEs might take longer, as it runs data from the beginning, while subqueries not.

Step 3:

My biggest challenges when replacing my subqueries with CTEs were the complexity of the query itself. As it started to have many variables, columns, joins etc, I would frequently get lost while writing it. Also, the constant change on tables would sometimes break my query as I would not refer to them correctly and systematically. Overall it was a good learning experience but it will require a lot more practice to become proficient on this task.